

Product Requirements Document

Hana Hideaway B&B — Hawaii Hospitality Dashboard & Marketing Website

Field	Value
Project Name	Hana Hideaway B&B; — Hawaii Hospitality Dashboard
Student	Noah Munz
Island	Maui, Hawaii
Target Segment	Eco-tourists
GitHub URL	github.com/Noahmunz21/ics385spring2026
Document Version	v3 — May 1, 2026
Course	ICS 385 Spring 2026 University of Hawaii Maui College

§1. Problem Statement & Target User

Small locally-owned hospitality businesses on Maui lack affordable, professional web presences that simultaneously market to visitors, surface real Hawaii tourism data, and give property owners a secure way to manage their listing. Hana Hideaway B&B addresses this gap with a three-page full-stack application: a public marketing page, an interactive visitor-statistics dashboard, and a password-protected admin panel.

The primary target user is an eco-tourist (Maya, 31) browsing on mobile who wants authentic, sustainable accommodation. The secondary user is the property admin (Kai, 45) who needs a simple, secure portal to post offers and monitor occupancy trends — without writing any code.

§2. Functional Requirements

ID	Tag	Requirement
R1		Public marketing page renders Hero, About, Amenities, and CTA sections, fetching property data from the Express API.
R2		Visitor Statistics Dashboard displays three Chart.js visualizations: monthly arrivals (bar), visitor origin (donut), and KPI metric cards.
R3		OpenWeatherMap widget shows live weather for the selected island; island selector dropdown updates all charts and weather simultaneously.
R4		Express REST API exposes GET /properties, GET /properties/:id, and POST /properties/:id/reviews backed by MongoDB Atlas.
R5		Admin login page (/admin/login) accepts email + bcrypt-hashed password via Passport.js LocalStrategy.
R6	[NEW v3]	Google OAuth 2.0 sign-in via passport-google-oauth20; users provisioned with googleId, displayName, email, provider='google'.
R7	[NEW v3]	Account linking: match existing user by googleId, then by verified email, otherwise create new document.
R8		Protected admin dashboard (/admin/dashboard) displays property list, review counts, and logged-in admin email; accessible only when authenticated.
R9		Logout destroys server-side session and redirects to /admin/login.
R10	[NEW v3]	All secrets stored in .env (git-ignored); .env.example committed with placeholder values.

§3. Technical Architecture

Stack: Node.js v18+ · Express.js · EJS (server-side) + React + Vite (client-side) · MongoDB Atlas + Mongoose · Passport.js (local + google) · Chart.js via react-chartjs-2 · OpenWeatherMap API · bcrypt · express-session · dotenv

Request Flow:

- Browser sends request → Express router matches route.

- If protected: isAuthenticated() / ensureAuth() middleware checks session → redirect to /admin/login if unauthenticated.
- If API route: Mongoose queries MongoDB Atlas → JSON response.
- If admin/Google OAuth: Passport strategy verifies credentials → serializeUser stores user ID in session.
- React frontend fetches /properties/:id via useEffect → renders components with props.

§4. Authentication & Security

Q1 — Authentication Strategies

Local Strategy (Passport.js): Email + bcrypt-hashed password (10 salt rounds). Ideal for the property admin who manages the system regularly and prefers a dedicated credential. **Google OAuth 2.0 (passport-google-oauth20):** Federated sign-in via Google account. Eliminates password management entirely for users who prefer Google — reducing credential-fatigue and phishing risk for the target eco-tourist segment.

Q2 — User Data Model

Field	Type	Constraints	Notes
email	String	required, unique, lowercase	Used for local login lookup
password	String	nullable	bcrypt hash; null for OAuth-only users
googleId	String	sparse unique index	Google profile ID; null for local users
displayName	String	required	From Google profile or registration
provider	String	default: 'local'	'local' or 'google'
role	String	default: 'admin'	Future: 'user' for guests
createdAt	Date	default: Date.now	Auto-set on document creation

Q3 — Account Linking

When a Google OAuth login arrives: (1) search for existing user by googleId — if found, return user. (2) If not found, search by verified email — if found, link googleId to existing account. (3) If neither match, provision a new user document with googleId, email, displayName, and provider='google'.

Q4 — Session Management

Sessions use express-session with a connect-mongo store backed by MongoDB Atlas. Cookie attributes: httpOnly: true (prevents JS access), maxAge: 86400000 (24-hour TTL for development; 14-day for production), secure: true in production (HTTPS only), sameSite: 'lax'. Sessions are destroyed on logout via req.logout() + req.session.destroy().

Q5 — Secret Storage

Environment Variable	Purpose
MONGODB_URI / MONGO_URI	Atlas connection string including credentials
SESSION_SECRET	Signs and verifies express-session cookies
GOOGLE_CLIENT_ID	OAuth 2.0 client identifier from Google Cloud Console
GOOGLE_CLIENT_SECRET	OAuth 2.0 client secret — never committed to GitHub
VITE_WEATHER_KEY	OpenWeatherMap API key for weather widget

All variables stored in .env (git-ignored). .env.example committed with placeholder values. dotenv loaded at process start via require('dotenv').config().

Q6 — Protected Routes

Route	Method	Guard Middleware	Redirect if Unauth
/admin/dashboard	GET	isAuthenticated()	/admin/login
/admin/logout	GET	isAuthenticated()	/admin/login
/profile	GET	ensureAuth()	/

/auth/google/callback	GET	Passport handles	/
-----------------------	-----	------------------	---

Q7 — OWASP Top-10 Threat Mitigation

OWASP Category	Threat	Control in This App
A01 Broken Access Control	Unauthenticated user accesses /admin/dashboard	isAuthenticated() / ensureAuth() middleware enforces 302 redirect on every protected route
A02 Cryptographic Failures	Plain-text passwords stored in database	bcrypt with 10 salt rounds via Mongoose pre-save hook; passwords never logged or returned in API responses
A07 Auth Failures	Brute-force or credential-stuffing attacks	Generic error messages (no user enumeration); Google OAuth offloads credential management to Google; session secrets rotated via env vars

§5. Acceptance Criteria & Test Scripts

All criteria use Given–When–Then (GWT) format. AC-3 through AC-7 have corresponding automated Jest + Supertest tests.

AC-1 — Public Marketing Page

GIVEN any visitor / WHEN they request GET / / THEN the marketing page returns HTTP 200 and renders the property name, hero image, and at least 3 amenities.

AC-2 — Visitor Dashboard

GIVEN any visitor / WHEN they request GET /dashboard / THEN three Chart.js visualizations are rendered with non-empty data and the weather widget displays current temperature.

AC-3 — Local Sign-Up

GIVEN a new email / WHEN POST /register with valid email and password ≥ 8 chars / THEN HTTP 302 to /admin/dashboard, the users collection contains the new document, and the password field is a bcrypt hash starting with \$2b\$.

AC-4 — Local Sign-In

GIVEN a registered user / WHEN POST /admin/login with correct credentials / THEN HTTP 302 to /admin/dashboard and a session cookie is set in the response headers.

AC-5 — Google OAuth Sign-In

GIVEN any user with a Google account / WHEN they click 'Sign in with Google' and grant consent / THEN HTTP 302 to /profile, and the users collection contains a document with googleId and provider='google'.

AC-6 — Protected Route Guard

GIVEN an unauthenticated visitor / WHEN they request GET /admin/dashboard / THEN HTTP 302 redirect to /admin/login and no admin content is rendered.

AC-7 — Logout

GIVEN an authenticated user / WHEN they POST /logout / THEN the session cookie is cleared, req.user is undefined on the next request, and the user is redirected to /admin/login.

AC-8 — Secret Hygiene

GIVEN the deployed GitHub repository / WHEN inspecting all committed files / THEN .env is absent, .env.example is present, and no API keys or secrets appear in any committed file.

Automated Test Scripts (Jest + Supertest)

File: tests/auth.test.js

AC-3 — Local Sign-Up

```
const request = require('supertest');
const app = require('../server');

test('AC-3: POST /register creates hashed user', async () => {
  const res = await request(app)
    .post('/register')
    .send({ email: 'test@example.com', password: 'TestPass123' });
  expect(res.statusCode).toBe(302);
});
```

AC-4 — Local Sign-In

```
test('AC-4: POST /admin/login with correct credentials redirects to dashboard',
  async () => {
    const res = await request(app)
      .post('/admin/login')
      .send({ email: 'admin@hanahideaway.com', password: 'HanaAdmin2026' });
    expect(res.statusCode).toBe(302);
    expect(res.headers.location).toBe('/admin/dashboard');
  });
```

AC-6 — Protected Route Guard

```
test('AC-6: GET /admin/dashboard redirects unauthenticated to /admin/login',
  async () => {
    const res = await request(app).get('/admin/dashboard');
    expect(res.statusCode).toBe(302);
    expect(res.headers.location).toBe('/admin/login');
  });
```

§6. Progress & Remaining Work

Week	Status	Deliverable
10	■ Complete	Mongoose Property schema, 5 seeded records, MongoDB Atlas
11	■ Complete	Express REST API, EJS template, Postman collection
12	■ Complete	React marketing page — Hero, About, Amenities, CTA
13	■ Complete	Chart.js dashboard, OpenWeatherMap widget, Island Cards
14	■ Complete	Passport.js local auth, bcrypt, admin dashboard, seed-admin.js
15	■ Complete	Google OAuth 2.0, PRD v3, Jest test scripts
16	■ Planned	Live demo, code walkthrough, deployment to Render

§7. AI Attribution

Claude (Anthropic) assisted with: Passport.js LocalStrategy and GoogleStrategy configuration, bcrypt pre-save hook implementation, connect-mongo session store debugging, Chart.js component structure, EJS template design, Jest + Supertest test scaffolding, and iterative PRD formatting across all three versions. All design decisions, property concept, user personas, and acceptance criteria were done by me with assistance from AI (Claude - Anthropic) to improve and iterate ideas/concepts further.